FlytBase Interview Prep

DevSecOps — Complete Answer Guide

SECTION 1 — CI/CD + DevSecOps

Walk me through your CI/CD pipeline end-to-end.

I designed a pipeline with 5 stages: source trigger → build & unit tests → SAST (Semgrep/SonarQube) → container build + image scan (Trivy) → staging deploy + DAST (OWASP ZAP) → prod deploy with approval gate.

Each stage has a quality gate. A failed gate blocks the pipeline — not just flags. Artefacts are signed and stored in a private registry.

> **SIGNAL TO SEND**
>
> Use 'I built' and 'I enforced' — not 'the team.' Own every decision.

Where exactly do you integrate SAST/DAST and why?

SAST runs post-build, pre-container — catches code-level issues (injection, hardcoded secrets) before they're baked into an image. Tool: Semgrep or SonarQube.

DAST runs against a live staging environment after deploy — catches runtime issues like auth bypass, missing headers, open redirects. Tool: OWASP ZAP or Burp Suite CI mode.

> **WHY IT MATTERS**
>
> Shift-left for SAST (cheap to fix) + runtime reality-check with DAST. Both are needed — neither alone is sufficient.

What if a vulnerability is detected — what happens next?

Pipeline halts. A ticket is auto-created (Jira/GitHub Issues) with severity, CVE ID, affected component, and remediation link. For CRITICAL/HIGH — the developer must fix and re-run. MEDIUM — can be deferred with written justification. LOW — tracked, not blocking.

I also send an automated Slack alert to the team channel so nothing is silently buried.

> **FOLLOW-UP: 'WHAT IF THIS SLOWS DEPLOYMENTS?'**
>
> Say: 'It does add ~4 minutes. I optimized by running SAST and unit tests in parallel, and DAST only on changed services. The cost of a breach is orders of magnitude higher than 4 minutes.'

How do you enforce zero-CVE or security gates?

I define policy-as-code — using OPA/Conftest or native GitHub Actions policy steps. Rules: no CRITICAL CVEs in base images (Trivy gate), no secrets in code (GitLeaks pre-commit hook), no unapproved base images (allowlist in registry).

These run on every PR, not just main. Zero-CVE is a target state, not a one-time achievement — we track drift weekly via dashboards.

SECTION 2 — AWS + Architecture

Design a secure AWS architecture for a microservices app.

VPC with public subnets (ALB only) and private subnets (ECS/EKS workloads, RDS). Traffic flows: Internet → WAF → ALB → private subnet services. No services have public IPs.

Security layers: WAF for OWASP rules, Security Groups (least privilege, no 0.0.0.0), NACLs for subnet-level control, VPC Flow Logs → CloudWatch, GuardDuty for threat detection, Secrets Manager for credentials.

> **KEY MENTION**
>
> Enable VPC endpoints for S3/DynamoDB to avoid traffic leaving the AWS backbone. Enable CloudTrail across all regions.

How do you isolate environments (dev, staging, prod)?

Separate AWS accounts per environment (AWS Organizations + SCPs). This is the strongest isolation — a compromised dev account can't touch prod at all.

Each account has its own VPC, IAM roles, and ECR registry. Cross-account deployments happen via role assumption with time-limited credentials — never shared long-lived keys.

How do you implement least privilege in IAM?

Start with deny-all, add only required actions per service. Use IAM Access Analyzer to detect over-permissive policies. All CI/CD roles use OIDC — no static access keys. Human access via SSO only (no console IAM users).

I audit unused permissions monthly using IAM Access Advisor and revoke anything not used in 30+ days.

> **TWIST ANSWER: BIGGEST AWS SECURITY MISTAKE?**
>
> 'Over-permissive IAM. People attach AdministratorAccess to Lambda functions or ECS task roles for convenience. That single mistake can turn a code injection into a full account compromise.'

SECTION 3 — Kubernetes + Containers

How do you secure a Kubernetes cluster?

Control plane: enable RBAC (no cluster-admin for apps), audit logging, API server restricted to VPN/bastion. Workloads: non-root containers, read-only root filesystem, drop all capabilities, no privileged pods (enforced via OPA Gatekeeper or Kyverno).

Network: NetworkPolicies to isolate namespaces (default deny-all). Secrets: use AWS Secrets Manager via External Secrets Operator — never env vars with raw secret values. Images: only pull from approved private registry.

How do you ensure zero downtime deployments?

Rolling deployments with maxUnavailable: 0 and maxSurge: 1. Readiness probes must pass before traffic is shifted — this is the critical gate. Pre-stop lifecycle hooks for graceful drain. PodDisruptionBudgets to ensure minimum replicas stay healthy during node drain.

For high-risk changes: blue/green via separate deployments + service selector swap, or canary via Argo Rollouts with automated metric-based promotion.

A pod keeps crashing — debug it step by step.

1. kubectl get pods -n — note restart count and status (CrashLoopBackOff, OOMKilled, Error).

2. kubectl describe pod — check Events section for scheduling issues, image pull errors, probe failures.

3. kubectl logs --previous — get logs from the crashed container.

4. Check resource limits — OOMKilled means memory limit too low. Check liveness probe config — too aggressive probes kill healthy pods on startup.

5. If image issue: verify digest, check registry credentials. If config issue: validate ConfigMap/Secret mounts.

SECTION 4 — Terraform + IaC

How do you structure Terraform for multiple environments?

Module-based structure: reusable modules in /modules (vpc, eks, rds), environment-specific configs in /envs/dev, /envs/prod. Each env calls the same modules with different variable files (terraform.tfvars).

Separate state per environment. Use Terragrunt to reduce repetition — it handles remote state config, dependency ordering, and environment-specific overrides cleanly.

How do you manage Terraform state securely?

Remote state in S3 with versioning enabled + DynamoDB table for state locking (prevents concurrent applies). S3 bucket: SSE-KMS encryption, block all public access, access restricted to CI/CD role only via bucket policy. State never stored locally or in git.

> **NEVER SAY**
>
> 'We just use local state' or 'state is in the repo' — these are immediate red flags for production experience.

What's your rollback strategy if infra breaks?

First: don't delete, revert. All infra changes via PR — a bad apply means reverting the PR and re-running. State is versioned in S3 so I can restore a previous state file if needed.

For production: I run terraform plan in CI on every PR — humans review the plan before apply. Sentinel or OPA policies block destructive changes (e.g. -/+ resource replacements on databases).

SECTION 5 — Observability + SRE

How do you define and track SLOs?

SLO = SLI target over a time window. Example: '99.9% of API requests respond in under 300ms, measured over a 30-day rolling window.' SLI is the actual measured metric (latency p99 from Prometheus). Error budget = 1 - SLO target.

I track error budget burn rate — if burn rate exceeds 5x normal, we page. If budget is nearly exhausted, we freeze feature deploys and focus on reliability. Dashboards show burn rate, not just uptime.

What's your monitoring stack and why?

Metrics: Prometheus + Grafana. Logs: Loki or ELK. Traces: Jaeger or AWS X-Ray. Alerts: PagerDuty with escalation policies. For Kubernetes: kube-state-metrics + node-exporter for cluster health.

I instrument applications with OpenTelemetry — vendor-neutral, so we can swap backends. Every service emits the four golden signals: latency, traffic, errors, saturation.

> **FOLLOW-UP: ALERTS VS DASHBOARDS?**
> 'Alerts are for humans at 2 AM — they must be actionable or they become noise. Dashboards are for investigation. I'd rather have 5 high-quality alerts than 50 that nobody trusts.'

Can monitoring be proactive instead of reactive? (AI/Predictive angle)

'Yes — I use anomaly detection on metrics. Tools like CloudWatch Anomaly Detection or Grafana ML models establish a baseline and alert on deviations, not just threshold breaches. This catches gradual degradation before it becomes an outage.'

I've also used predictive autoscaling — feeding historical traffic patterns to scale ahead of load spikes rather than reacting to them. For disk/memory, trending analysis (will this fill in 6 hours?) gives ops teams time to act proactively.

> **AI ANGLE — FLYTBASE BONUS**
> Mention LLM-assisted runbooks: 'I explored using AI to correlate multi-signal alerts and suggest probable root cause — reducing the cognitive load during incidents.'

SECTION 6 — Scenario-Based

Production is down. What do you do in the first 10 minutes?

Min 0–2: Confirm the incident is real (not a monitor flap). Check status page, try manually, check 2 independent signals. Declare incident and open war room (Slack/bridge). Assign incident commander.

Min 2–5: Check recent changes — last deploy, config change, infra event. 80% of outages are caused by recent changes. If a deploy happened: rollback immediately, ask questions later.

Min 5–10: Check dashboards — which services are erroring? Check error logs for common exception. Communicate status to stakeholders — even 'we're investigating' is better than silence.

> **KEY PHRASE**
>
> 'I bias toward rollback first — restore service, then diagnose. User impact stops; root cause analysis can happen calmly afterward.'

Deployment failed at 2 AM, no one is online. What happens?

'This shouldn't be a crisis — I design pipelines to be safe by default. If a deployment fails, the pipeline halts. The previous version is still running. No action needed immediately.'

Automated rollback kicks in if health checks fail post-deploy (Argo Rollouts or ECS circuit breaker). On-call engineer is paged if SLOs breach. The failed deploy is flagged for review in the morning with full logs retained.

Traffic suddenly increases 10x. What breaks first?

Database connections — most apps don't pool aggressively. Connection pool exhaustion → cascading timeouts. Second: any service without HPA or with memory limits too tight → pods OOMKilled under load.

I address this proactively: connection pooling via PgBouncer, HPA on CPU+RPS metrics, pre-warming during known traffic events, load testing at 2x–5x normal traffic to find the breaking point before users do.

A container image has a critical CVE in production. What now?

Triage first: is this CVE exploitable in our context? (Severity ≠ exploitability — some CVEs are in unused code paths.) Check CVSS vector — network exploitable with no auth is highest priority.

If exploitable: immediate remediation — update base image, rebuild, redeploy. If patch not available: mitigate at network level (WAF rule, NetworkPolicy to isolate pod). Notify security team. Log everything for audit.

Prevent recurrence: add this CVE to the pipeline gate so it's blocked from future deploys.

SECTION 7 — Behavioral + Trap Questions

A developer wants to bypass security checks to ship faster — what do you do?

'I don't frame it as security vs speed — that's a false tradeoff. I ask: what's the actual blocker? Usually it's a slow scan or a false positive. I fix that.'

If it's a real vulnerability they want to skip: I explain the risk in business terms — a breach costs 100x more than a delayed ship. If they still push: I escalate to the CISO/engineering lead. I document my objection.

> **WHAT THEY'RE TESTING**
> Do you cave under pressure? Do you own security even when it's uncomfortable? Say 'I enforced' not 'I tried to suggest.'

Would you ship fast or wait for perfection?

'Ship fast with guardrails. Perfection is a moving target. But my pipelines enforce non-negotiable baselines automatically, so the developer doesn't have to think about security on every commit — the system catches it.'

Bias to action + quality gates = neither extreme.

■ Trap: 'Do you know everything about Kubernetes?'

> **NEVER SAY**
> 'Yes, I know everything' — arrogance red flag. Or 'No, not really' — confidence killer.

> **SAY THIS**
> 'Not everything — it's a vast ecosystem. But I'm confident diagnosing real issues in production. When I hit something new, I know where to look and how to learn fast. What I don't know today, I can figure out by tomorrow.'

■ Trap: 'Have you worked in production?'

> **NEVER SAY**
> 'I was just an intern' or 'not really, mostly dev/staging.'

> **SAY THIS**
> 'Yes — I worked on production-grade systems with real SLOs, real incident response, and real security constraints. The systems I built handled actual traffic with availability targets we were accountable for.'

■ Trap: 'What are your weaknesses?'

NEVER SAY

'I'm a perfectionist' or 'I work too hard.'

SAY THIS

Pick a real but non-critical gap: 'I'm still deepening my knowledge in eBPF-based security tooling like Cilium. It's an area I'm actively learning because I see it becoming essential for Kubernetes network observability.'

You don't know a tool they use — what do you do?

'I'd be upfront: I haven't used X specifically, but I've used Y which solves a similar problem. I pick up tools fast — give me a week and I'll be productive, give me a month and I'll be contributing improvements.'

Then show curiosity: 'What do you like about it? What problems does it solve for your team?' — this turns a weakness into a conversation and shows you care about their stack.

HOW THEY EVALUATE YOU

Trait	Signal they want
Ownership	'I built / I enforced / I designed'
Speed	You don't overthink — give direct answers
Depth	You explain WHY, not just WHAT
AI mindset	You think ahead of current DevOps
Clarity	No rambling — structured, concise

Good luck. You've done the prep — now own the room.